Day 05 — Ethereum & the EVM

Ethereum executes bytecode in a replicated state machine. Every operation consumes gas, and every node must reach the same result.

Accounts and transactions

EOAs hold ETH, have a nonce, and contain no code. Contract accounts contain bytecode and storage. A transaction pays a base cost and execution gas; chain IDs provide replay protection.

Opcode execution

The mini-EVM in this lesson models a bounded stack, memory, storage, jumps, return data, and errors. PUSH data is parsed before jump destinations are found. Unknown input, stack underflow, invalid jumps, and out-of-gas execution fail explicitly.

Storage and gas

Storage is addressed by 32-byte slots. Packed values share a slot; mappings use `keccak256(key, slot)`; dynamic arrays derive element locations from the hashed base slot. The first access to a storage key is cold and later accesses are warm in the same transaction.

Security lessons

- Bound gas limits and trace memory.
- Apply EIP-2200 storage rules and refund caps.
- Match the reference EVM with differential tests.
- Reject malformed values instead of guessing.

Practice

Complete the exercises in `exercises/README.md`: unpack a slot, observe warm reads, and extend the differential opcode test.